

Manual Técnico — Gestão MAC Mobile (UniFi) — v4

Documento de referência para entender, manter e evoluir o sistema.

Novidades da v4: espelho do log nativo da UniFi (tabela `unifi_audit`), auto-atualização distribuída com `*lease*` (só um terminal coleta por janela), presença de usuários (■ N conectados via `/api/ping`), app **silent** (sem console) que encerra ao fechar o navegador (watchdog), pasta `logs/` (rotação diária), `secret.key` na **raiz** (mesma chave por PC, DB na rede) e trava de escrita por WLAN em `add/remover/troca`. Versionado no Git (tag `v4`).

Para uso diário veja `COMO_RODAR.md`; para rede/multiusuário veja `REDE_MULTIUSUARIO.md`; para o estado/decisões veja `ESTADO_DO_PROJETO.md`.

1. O que é

Sistema web (rodando como `.exe` que abre o navegador) para monitorar e gerenciar os MACs das redes "mobile" (Wi-Fi) **de um controller** UniFi OS (multi-site). Cada site tem uma WLAN mobile com allow-list limitada a 512 MACs. Host, credenciais e nomes dos sites vêm do ambiente (não ficam no código).

Objetivos:

- Coletar e **entender uso x desuso** dos MACs (regra à prova de férias).
- Manter **cadastro** de cada MAC (pessoa, setor, unidade, líder, gestor, chamado, termo, VIP).
- **Auditoria** de tudo (removido, voltou, bloqueado, troca, ações manuais...).
- Ações manuais **unitárias** (adicionar/remover 1 MAC, troca de aparelho).
- **Backup** e operação **multiusuário** com banco em pasta de rede.

2. Stack e técnicas usadas

- **Python 3.12 + Flask** (servidor web local).
- **SQLite** (banco único em arquivo; `unifi/db.py`). Sem ORM — SQL direto.
- **requests** (API do UniFi OS, com sessão + token CSRF).
- **openpyxl** (importação de planilhas `.xlsx`).
- **cryptography (Fernet)** (criptografia da senha do UniFi no banco).
- **PyInstaller** (gera o `.exe` único) + **webbrowser** (abre o navegador).
- **Pillow** (gera o ícone `.ico` a partir do logo).
- Front: HTML (Jinja2) + CSS próprio (tema escuro, gráficos em CSS puro, responsivo). Sem framework JS.

Técnicas-chave:

- **API UniFi OS:** login em `/api/auth/login`; dados em `/proxy/network/api/s/<site_id>/...`; token CSRF capturado do header/JWT.
- **Somente leitura por padrão + escrita controlada:** a camada HTTP (`UnifiClient._request`) só permite **GET** e **PUT** (PUT só para editar a `allow-list` em `rest/wlanconf`). POST/DELETE (`block-sta` etc.) são bloqueados.
- **Merge histórico:** cada coleta atualiza o "maior `last_seen` já visto" por (`site`, `MAC`) — memória própria, independente do controller.
- **Regra dos 35 dias (à prova de férias):** um MAC só é "disponível/liberável" após >35 dias sem logar (`AVAILABLE_DAYS`). `NEVER_MODE=grace` protege quem nunca conectou (só conta após 35 dias da 1ª coleta).
- **Concorrência (rede SMB):** `busy_timeout=20s`, sem WAL; travas de edição (`edit_locks`) para avisar uso simultâneo.
- **Autenticação = conta do UniFi** (validada no controller a cada login).

3. Arquitetura (arquivos)

<code>app.py</code>	Flask: rotas, login, sessão, coleta, ações manuais
<code>iniciar.py</code>	Entry do <code>.exe</code> : sobe o Flask e abre o navegador
<code>desktop.py</code>	(alternativo) abre em janela nativa (<code>pywebview</code>)
<code>collect.py</code>	Coleta 1x e grava no banco (para agendar)
<code>importar.py</code>	Importa planilha <code>.xlsx</code> (<code>dry-run / --apply</code>)
<code>cli.py</code>	CLI somente leitura (<code>sites</code> , <code>list</code>)
<code>build_exe.ps1</code>	Gera <code>dist\GestaoMAC.exe</code> (PyInstaller + ícone)
<code>unifi/</code>	
<code>client.py</code>	Cliente da API UniFi OS (GET + PUT da <code>allow-list</code>)
<code>inventory.py</code>	Coleta detalhada (<code>snapshot_all/snapshot_site</code>)
<code>db.py</code>	SQLite: schema, merge, classificação, clientes, auditoria, settings, locks, backup
<code>config.py</code>	Resolve credenciais do UniFi (do banco; semeia do <code>.env</code>)
<code>secret.py</code>	Criptografia (Fernet) da senha do UniFi
<code>templates/</code>	<code>base</code> , <code>login</code> , <code>index</code> , <code>overview</code> , <code>dashboard</code> , <code>clientes</code> , <code>cliente</code> , <code>adicionar</code> , <code>remover</code> , <code>auditoria</code> , <code>config</code> , <code>backup</code>
<code>static/style.css</code>	tema + gráficos + responsivo
<code>data/history.db</code>	banco (gerado). <code>data/secret.key</code> chave de cripto.

4. Banco de dados (SQLite) — tabelas

- **collections** (`id`, `ts`) — cada coleta.
- **mac_state** (PK `site_id+mac`): estado por MAC/site — `name`, `hostname`, `oui`, `blocked`, `in_allow_list`, `last_seen`, `last_online`, `first_seen`, `first/last_collected`.
- **seen_history** — histórico bruto por coleta (`online`, `last_seen`).
- **events** (`id`, `ts`, `site_id`, `site_desc`, `mac`, `event`, `detail`) — **auditoria**.
`event` ∈ {`cadastrado`, `voltou`, `removido`, `vip_removido`, `bloqueado`, `desbloqueado`, `add_manual`, `remove_manual`, `troca`}.
- **client_info** (PK `mac`): cadastro — `nome`, `setor`, `unidade`, `funcao`, `lider`,

gestor_autorizou, chamado, notes, termo, vip, created/updated.

- **settings** (key,value): login/credenciais (unifi_host/site/username, unifi_password_enc, unifi_verify, flask_secret).
- **edit_locks** (mac, who, ts): aviso de edição simultânea.

Migrações: `db._migrate()` adiciona colunas novas com `ALTER TABLE ... IF` (via `PRAGMA table_info`) — seguro em bancos antigos. Toda mudança de schema deve ir no `SCHEMA` (novos bancos) e no `_migrate` (bancos existentes).

5. Fluxo de coleta (read)

`inventory.snapshot_all(client)` percorre todos os sites → para cada WLAN mobile lê a allow-list + stat/alluser (last_seen/blocked) + stat/sta (online) → `db.record_snapshot()` mescla no banco e **detecta transições** (gera eventos).
A web coleta ao abrir (throttle 120s, se `COLLECT_ON_OPEN=1`) e o botão "Atualizar status" força. Para histórico contínuo: `collect.py` agendado.

6. Segurança

- **Login**: usuário/senha do UniFi, validados no controller. Sessão Flask (`flask_secret` persistido no banco). Sem conta separada.
- **Senha do UniFi**: cifrada (Fernet) em `settings.unifi_password_enc`; chave em `data/secret.key` (nunca no código/git). Salva no 1º login; muda só na tela ■.
- **Escrita controlada**: só edição da allow-list (PUT). Remoção bloqueia VIP.

7. Multiusuário / banco em rede

Ver `REDE_MULTIUSUARIO.md`. Resumo: `DB_PATH` aponta para `\\SERVIDOR\...\data\history.db` (e `secret.key` na mesma pasta); `COLLECT_ON_OPEN=0` nos PCs de usuário e 1 num coletor central; concorrência via `busy_timeout` + `edit_locks`.

8. Como rodar / buildar

- Dev: `.\venv\Scripts\python.exe app.py` → `http://127.0.0.1:5000`
- Build do exe: `.\build_exe.ps1` → `dist\GestaoMAC.exe` (preserva `dist\data`).
- Dependências: `requirements.txt` (+ `pyinstaller/pywebview/pillow` só p/ build).

9. ■ COMO ADICIONAR NOVOS SITES (no futuro)

Sites são descobertos automaticamente do controller (`UnifiClient.get_sites` → `/proxy/network/api/self/sites`). Então:

1. **Crie o site normalmente na UniFi** e configure a WLAN mobile (nome contendo

"MOBILE" — o sistema detecta por isso, em `get_mobile_wlans`).

2. **Rode uma coleta** (abra o app/Atualizar status, ou `collect.py`). O novo site **já aparece** em Sites, Visão Geral, dashboards, etc. Nenhuma mudança de código é necessária.

Ajustes **opcionais**:

- **Número de unidade no checklist do cadastro**: se o site novo tem um número de unidade novo (ex.: 120), adicione-o em `UNITS` (variável de ambiente no `.env`):
`UNITS=101,102,...,117,120`. (Padrão no `app.py`)
- **Importar planilha do site novo**: o mapa "nome da aba → número da unidade" fica em `sites_map.json` (raiz do projeto, arquivo LOCAL não versionado). Adicione lá a entrada em `sheet_unit` e o número em `valid_units`. (Há um exemplo genérico embutido como fallback em `unifi/sheet_import.py`)
- **Nome da WLAN diferente**: se a WLAN mobile do site novo **não** tiver "MOBILE" no nome, ajuste `UnifiClient.get_mobile_wlans` (`unifi/client.py`) para reconhecê-la (ex.: incluir outro termo).

10. Como estender (receitas rápidas)

- **Novo campo de texto no cadastro**: adicione em `db.CLIENT_FIELDS`, na coluna da tabela `client_info` (`SCHEMA + _migrate`), e um `<input name="...">` em `templates/cliente.html`. Aparece sozinho em busca/backup.
- **Novo campo "check" (booleano)**: coluna na `client_info` (`SCHEMA+migrate`), função `set_x` (espelhe `set_vip`), checkbox no template e `db.set_x(...)` no POST de `cliente()` em `app.py`.
- **Novo tipo de evento na auditoria**: adicione em `db.EVENT_LABEL` e gere com `db.add_event(...)` (ou na detecção em `record_snapshot`). Cor opcional em `static/style.css` (`.pill.ev-<tipo>`).
- **Mudar a regra de dias**: `db.AVAILABLE_DAYS` (default 35) e os limiares em `_classify / overview_summary` (faixas `>50/>100`).

11. Observações

- O `.exe` é só leitura+ações unitárias; **não** faz alterações em massa.
- Backup: tela ■ Backup (`.db` consistente / `.csv`) — também salvo em `backups/`.
- Recomendado trocar a senha do UniFi periodicamente (a senha foi digitada no chat numa sessão de criação).